

Capitolo 33 — PROT-004 — Canonical Dispatch & Idempotency

Pubblico	Lettori tecnici e non tecnici del WCM Process & Memory Book
Versione	FROZEN-33
Data master	2026-08-30
Stato	FROZEN
Source path	wcm/documentation/process-memory-book/ chapters/33_prot_004_canonical_dispatch_idempotency.md
Source SHA	9578ec2
Release	2026-08-31T20:48:50.990Z

GitHub main è la source of truth. Questo PDF è una release derivata: non costituisce approvazione né autorità.

PARTE VII — Il Libro dei Protocolli WCM Stato: FROZEN **Data:** 2026-08-30 **Scope:** WCM generale, domain-agnostic

33.0 Consegnare un lavoro una volta sola, con il contesto giusto

In qualunque organizzazione esiste un momento delicato: quello in cui un lavoro passa da chi decide **che cosa deve essere eseguito** a chi dovrà materialmente eseguirlo.

Finché il lavoro resta nello stesso punto, l'ambiguità è relativamente facile da controllare. Quando attraversa un confine organizzativo, invece, possono comparire due problemi opposti.

Il primo è la **perdita di contesto**: chi riceve il lavoro sa di dover fare qualcosa, ma non riceve in modo affidabile l'identità del lavoro, la versione osservata, il luogo in cui operare o gli altri riferimenti necessari.

Il secondo è la **duplicazione**: lo stesso lavoro viene consegnato più volte, magari perché un controllo periodico continua a trovarlo disponibile e ogni volta genera una nuova attivazione.

PROT-004 — Canonical Dispatch & Idempotency nasce per governare precisamente questo confine.

La regola fondamentale può essere espressa così:

Prima di attivare un esecutore cognitivo per un lavoro operativo, deve esistere una consegna persistente, riconoscibile e deduplicabile di quel lavoro.

In termini semplici: non basta “svegliare qualcuno”. Bisogna consegnargli **un oggetto di lavoro durevole**, capace di dire quale lavoro è, dove si trova, quale versione è stata osservata e se quella stessa consegna è già avvenuta.

33.1 Il problema che PROT-004 risolve

Immaginiamo un esempio pedagogico.

Un ufficio riceve una richiesta che deve essere lavorata da uno specialista. Ogni cinque minuti un addetto controlla l'elenco delle richieste aperte. Se vede la richiesta ancora presente, chiama lo specialista.

Al primo controllo la chiamata è corretta.

Al secondo controllo la richiesta può risultare ancora aperta perché lo specialista la sta lavorando. Se l'addetto chiama di nuovo, lo stesso lavoro viene consegnato una seconda volta. Al terzo controllo può succedere ancora.

Il problema non si risolve semplicemente controllando meno spesso. Una frequenza più bassa riduce la probabilità di duplicazione, ma non stabilisce **se quella specifica richiesta sia già stata consegnata**.

Serve invece una traccia durevole del passaggio:

```
RICHIESTA PRONTA
  ↓
CONSEGNA PERSISTENTE
  ↓
PRESA IN CARICO
  ↓
ESECUZIONE
```

Quella consegna persistente è ciò che PROT-004 chiama **durable canonical dispatch**.

“Durable” significa che non vive soltanto nella memoria momentanea di una singola esecuzione. “Canonical” significa che la sua struttura rende riconoscibili gli elementi necessari al lavoro. “Dispatch” significa che rappresenta il passaggio operativo verso l'esecutore.

33.2 Control plane ed execution plane, senza gergo inutile

Il protocollo distingue due ruoli logici.

Il **control plane** è la parte del sistema che stabilisce se esiste lavoro eleggibile da consegnare e a chi deve essere indirizzato.

L'**execution plane** è la parte che prende in carico quel lavoro e lo esegue.

Non è necessario immaginare due applicazioni separate. La distinzione è funzionale.

Un esempio quotidiano può aiutare:

- il banco accettazione di un laboratorio riceve e registra una richiesta;
- il tecnico del laboratorio esegue materialmente l'analisi.

Il banco accettazione non svolge l'analisi; il tecnico non dovrebbe inventare da solo quale richiesta lavorare. Tra i due serve un passaggio affidabile.

Nel WCM, PROT-004 governa quel passaggio quando un controllo deterministico deve attivare un'esecuzione cognitiva o comunque un esecutore che necessita di contesto operativo.

33.3 Il trigger

PROT-004 si attiva quando un lavoro operativo è realmente pronto per essere consegnato all'esecutore e il sistema sta per produrre l'attivazione corrispondente.

Il flusso concettuale è:

```
LAVORO READY
↓
DEVE ESSERE CONSEGNATO A UN ESECUTORE
↓
PROT-004
```

Il protocollo interviene **prima** che un semplice segnale di attivazione venga considerato equivalente alla consegna del lavoro.

Questo è un punto centrale: un segnale può dire “c'è qualcosa da fare”, ma non necessariamente trasferisce in modo affidabile tutto ciò che serve per sapere **che cosa**, **dove** e **su quale versione** lavorare.

33.4 Gli input necessari

Per costruire una consegna canonica e deduplicabile, il sistema deve disporre di elementi sufficienti a identificare il lavoro e il suo contesto.

La baseline corrente richiede che il dispatch renda disponibili o risolvibili almeno:

- identificatore del lavoro;
- path o riferimento persistente dell'oggetto operativo;
- progetto o perimetro organizzativo;
- branch o baseline operativa pertinente;
- versione remota osservata.

Quando applicabile devono inoltre essere coerenti:

- route o allowlist;
- assignee autorizzato;
- progetto/workspace di destinazione;
- eventuali gate di governance pendenti.

Il principio è semplice: **l'esecutore non deve ricostruire per supposizione quale lavoro gli sia stato assegnato.**

33.5 Il durable canonical dispatch

Il dispatch è un **envelope di lavoro persistente**.

La parola “envelope” può sembrare tecnica, ma l'idea è molto comune: una busta contiene ciò che serve per far arrivare qualcosa al destinatario corretto e per identificarne origine e destinazione.

Nel WCM l'envelope non deve necessariamente contenere ogni dettaglio del lavoro. Deve però contenere, o rendere risolvibili, i riferimenti sufficienti perché l'esecutore possa raggiungere la fonte autorevole corretta.

Questo evita due errori opposti.

Il primo sarebbe copiare dentro il dispatch tutta la conoscenza possibile, trasformandolo in una seconda source of truth.

Il secondo sarebbe renderlo così povero da non consentire all'esecutore di capire quale oggetto operativo debba aprire.

La funzione corretta è intermedia:

il dispatch trasporta identità, claim e contesto di instradamento; la source of truth del lavoro resta separata.

33.6 La source of truth non si sposta con il dispatch

PROT-004 stabilisce esplicitamente una separazione importante.

Il **Service Job persistente** resta il contratto operativo e la fonte di verità del suo stato.

Il **dispatch** rappresenta invece:

- la consegna durevole;
- la presa in carico;
- l'audit del passaggio;
- l'envelope di contesto.

In forma compatta:

```
SERVICE JOB
= contratto operativo + stato autorevole

DISPATCH
= consegna + claim + contesto + audit
```

Questa distinzione impedisce che il sistema inizi ad avere due versioni concorrenti dello stesso stato operativo.

Se il dispatch diventasse una seconda fonte di verità indipendente, potrebbero emergere domande difficili: quale stato prevale? cosa succede se uno dice **DONE** e l'altro no? quale dei due autorizza una nuova esecuzione?

PROT-004 evita il problema alla radice: il dispatch non sostituisce l'oggetto operativo autorevole.

33.7 Idempotenza: ripetere il controllo senza ripetere il lavoro

Una delle parole centrali del protocollo è **idempotenza**.

In linguaggio semplice, qui significa:

poter ripetere lo stesso controllo senza produrre ogni volta una nuova consegna equivalente.

Un sistema operativo può dover controllare periodicamente se esiste lavoro disponibile. La ripetizione del controllo è normale.

Ciò che non deve ripetersi automaticamente è la consegna dello **stesso lavoro nella stessa versione osservata**.

Per questo la baseline definisce una chiave logica costruita almeno su:

```
IDENTITÀ DEL LAVORO + VERSIONE REMOTA OSSERVATA
```

Nel protocollo canonico questa idea è rappresentata dallo schema:

```
WCM_DISPATCH_KEY=<SERVICE_JOB_ID>:<REMOTE_SHA>
```

Il dettaglio tecnico non è importante per comprendere il principio. Conta ciò che la chiave consente di domandare:

“Ho già una consegna aperta equivalente per esattamente questo lavoro e questa versione?”

Se la risposta è sì, un nuovo controllo deve poter terminare senza creare una seconda attivazione cognitiva.

33.8 Perché la versione fa parte dell'identità logica

Usare soltanto l'identificatore del lavoro non sarebbe sempre sufficiente.

Un lavoro può cambiare nel tempo. Può essere aggiornato, corretto o esplicitamente riaperto. La stessa identità organizzativa potrebbe quindi riferirsi a una versione operativa diversa.

Associare il dispatch anche alla versione remota osservata consente di distinguere:

```
LAVORO A – VERSIONE 1  
≠  
LAVORO A – VERSIONE 2
```

Questo non significa che ogni variazione autorizzi automaticamente una nuova esecuzione. Significa che il meccanismo di deduplicazione possiede una base più precisa per distinguere due stati remoti diversi.

La decisione di eleggibilità resta soggetta alle regole del processo operativo e alla governance applicabile.

33.9 Un lavoro, un dispatch logico

La prima invariante di PROT-004 è:

uno stesso lavoro, sulla stessa versione remota, non deve generare più dispatch cognitivi equivalenti.

Questo è diverso dal dire che nel sistema possa esistere un solo oggetto tecnico in assoluto.

L'invariante riguarda il significato operativo: non devono esistere più consegne aperte equivalenti che inducono più esecutori, o lo stesso esecutore più volte, a trattare lo stesso lavoro come se fosse nuovo.

La deduplicazione deve inoltre essere **persistente**.

Non basta dire:

“Non creo un secondo dispatch perché il primo processo è ancora in esecuzione.”

Se quel processo termina, si interrompe o viene dimenticato dal runtime, il sistema perderebbe la protezione.

La traccia della consegna deve sopravvivere al singolo run.

33.10 Il claim prima della duplicazione

Una volta materializzato un dispatch aperto equivalente, i controlli successivi devono poter riconoscere che quel lavoro è già stato consegnato.

Questa è la logica del **claim**.

Il claim non significa necessariamente che il lavoro sia già completato. Significa che il sistema possiede una traccia persistente del fatto che la consegna è stata materializzata e può quindi evitare di trattarla come nuova.

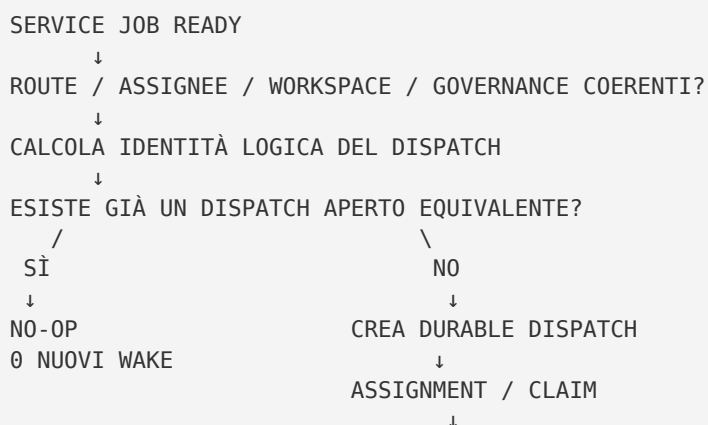
Un'analogia pedagogica è il numero progressivo consegnato allo sportello.

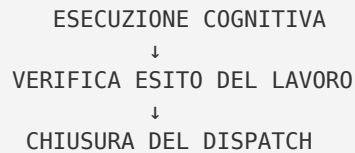
Se una pratica ha già il suo numero ed è stata assegnata, ristampare ogni minuto un nuovo numero per la stessa pratica non accelera il lavoro. Produce soltanto duplicati.

PROT-004 trasforma questa intuizione in una proprietà operativa verificabile.

33.11 Il flusso completo

Il protocollo può essere letto come una sequenza di gate.





La caratteristica importante è che il controllo può essere ripetuto molte volte senza trasformarsi automaticamente in molte esecuzioni cognitive.

Il **control loop** può quindi essere ricorrente; il **cognitive loop** viene attivato soltanto quando esiste lavoro reale non già consegnato in modo equivalente.

33.12 Il gate prima del dispatch

Prima di creare una nuova consegna, PROT-004 richiede di verificare almeno:

- il lavoro è realmente **READY**;
- route e allowlist sono coerenti;
- l'assignee è autorizzato;
- progetto o workspace sono quelli corretti;
- non esiste già un dispatch aperto equivalente;
- esiste un meccanismo idempotente utilizzabile;
- non è pendente un gate di governance che impedisca l'esecuzione.

Questi controlli impediscono che il dispatch venga interpretato come una scorciatoia capace di superare authority o governance.

Il fatto che un lavoro possa essere tecnicamente consegnato non significa che debba esserlo.

33.13 L'envelope minimo

La baseline corrente rende espliciti alcuni campi minimi dell'envelope.

Non è necessario che il lettore non tecnico memorizzi i nomi tecnici. È sufficiente comprenderne il significato:

```
origine della consegna
identità logica del dispatch
identità del lavoro
path del lavoro
progetto
branch
versione remota osservata
```

Il protocollo canonico li rappresenta, nel runtime corrente, con campi come:

```
WCM_WAKE_SOURCE
WCM_DISPATCH_KEY
SERVICE_JOB_ID
SERVICE_JOB_PATH
PROJECT
```

BRANCH
REMOTE_SHA

Questi campi non sono decorativi. Rendono il passaggio **ricostruibile**.

Un esecutore o un auditor deve poter rispondere a domande come:

- quale lavoro ha provocato questa attivazione?
- quale versione era stata osservata?
- quale percorso doveva essere usato?
- a quale perimetro apparteneva?
- questa consegna era nuova o equivalente a una già aperta?

33.14 Generic wake: svegliare non significa consegnare

Uno degli anti-pattern più importanti del protocollo è il **generic wake come work envelope**.

Un generic wake è un segnale generico che attiva un esecutore. Il problema nasce quando il runtime non garantisce che il payload applicativo associato al segnale arrivi realmente nel contesto dell'esecutore.

In quel caso può accadere qualcosa di paradossale:

```
IL SISTEMA SI SVEGLIA
      ↓
MA NON SA CON CERTEZZA QUALE LAVORO DEVE ESEGUIRE
```

L'attivazione è avvenuta, ma la consegna no.

PROT-004 stabilisce quindi che, in un runtime con questo limite, un generic wake non è sufficiente per trasferire il lavoro.

Serve un oggetto persistente che l'esecutore possa ricevere o risolvere in modo nativo.

33.15 Perché il polling non deve diventare lavoro cognitivo

Un altro anti-pattern è usare un modello cognitivo semplicemente per chiedergli periodicamente:

| *"C'è qualcosa da fare?"*

Se la risposta può essere ottenuta leggendo stati strutturati e applicando regole meccaniche, l'uso dell'LLM non aggiunge valore semantico.

La baseline separa quindi:

```
CONTROL LOOP
= controllo deterministico dell'eleggibilità
```


COGNITIVE LOOP
= esecuzione attivata quando esiste lavoro reale

Questa separazione riduce attivazioni inutili e rende più prevedibile il comportamento del sistema.

PROT-004 non afferma che ogni discovery possa essere deterministica in ogni contesto. Governa i flussi in cui l'eleggibilità e la deduplicazione sono già rappresentabili mediante stato strutturato e regole definite.

33.16 Il tempo non è una prova di unicità

Un modo intuitivo per limitare duplicazioni è introdurre un intervallo di tempo: per esempio, non inviare due attivazioni entro cinque minuti.

Può essere utile come protezione secondaria, ma non risolve il problema logico.

Dopo sei minuti, la stessa consegna potrebbe essere duplicata.

Per questo PROT-004 distingue una riduzione probabilistica del problema da una deduplicazione persistente.

COOLDOWN / DEBOUNCE
= limita la frequenza

DURABLE CLAIM
= identifica che quel lavoro equivalente è già stato consegnato

La seconda proprietà è quella necessaria per l'idempotenza del dispatch.

33.17 Lifecycle del dispatch

Il protocollo definisce un lifecycle raccomandato che permette di capire se una consegna è soltanto creata, è in lavorazione oppure è terminata.

La forma corrente è:

Dispatch creato	→ todo
Assignment run	→ in_progress
Service Job DONE	→ dispatch done
BLOCKED reale	→ blocked con causa/owner
Fallimento terminale	→ cancelled

La cosa più importante non è il nome dell'etichetta, ma la coerenza tra stato della consegna e stato reale del lavoro.

In particolare, un dispatch non deve essere lasciato **in_progress** se non esiste più una continuazione viva.

Allo stesso modo, **done** non può essere usato soltanto perché l'esecutore ha terminato una chiamata o prodotto un output qualsiasi.

33.18 Terminalità verificabile

La chiusura del dispatch è subordinata all'esito coerente del lavoro operativo.

PROT-004 stabilisce che il dispatch vada chiuso **done solo dopo** che il Service Job abbia raggiunto un esito coerente e siano stati verificati gli acceptance criteria applicabili.

Questo collega direttamente PROT-004 alla disciplina di **PROT-002 – Result Acceptance & Closure**.

In parole semplici:

consegnato non significa completato; eseguito non significa automaticamente accettato; accettato è ciò che consente la chiusura coerente.

Il dispatch documenta il passaggio e il claim, ma non può dichiarare riuscito un lavoro che la sua source of truth non considera realmente concluso.

33.19 Cosa succede se qualcosa si blocca

Un dispatch può incontrare un blocco reale.

In questo caso il protocollo non richiede di fingere che il lavoro sia ancora in esecuzione, né di ricreare continuamente nuovi dispatch.

Il lifecycle prevede uno stato **blocked** accompagnato da causa e owner quando la baseline operativa lo supporta.

La funzione è mantenere distinguibili almeno tre situazioni:

```
LAVORO CONSEGNATO E IN CORSO
≠
LAVORO CONSEGNATO MA BLOCCATO
≠
LAVORO TERMINATO
```

Questa distinzione aiuta anche i controlli successivi: un dispatch bloccato non deve essere ignorato come se non fosse mai esistito.

33.20 Failure mode principali

PROT-004 protegge il sistema da una serie di errori ricorrenti.

Duplicazione del dispatch

Lo stesso lavoro nella stessa versione viene consegnato più volte perché ogni controllo lo interpreta come nuovo.

Effetto: più attivazioni cognitive equivalenti, lavoro duplicato e maggiore difficoltà nel ricostruire quale esecuzione sia quella valida.

Payload perso

Il sistema genera un segnale di attivazione con un contesto che il runtime non trasferisce realmente all'esecutore.

Effetto: l'esecutore si attiva ma non possiede un envelope affidabile del lavoro.

Deduplicazione solo temporale

Cooldown o frequenza ridotta vengono scambiati per una garanzia di unicità.

Effetto: il duplicato viene rinviato, non impedito logicamente.

Dispatch scambiato per source of truth

Lo stato dell'envelope viene trattato come autorità indipendente sul lavoro.

Effetto: due rappresentazioni possono divergere e produrre stati concorrenti.

Dispatch **in_progress** senza continuazione viva

La consegna rimane apparentemente in lavorazione, ma non esiste più un esecutore che la stia portando avanti.

Effetto: il sistema può evitare nuove consegne senza avere, in realtà, una lavorazione attiva.

Chiusura prematura

Il dispatch viene marcato **done** prima della verifica dell'esito e dei criteri di accettazione.

Effetto: la consegna appare conclusa mentre il lavoro autorevole non lo è.

33.21 Un esempio pedagogico completo

Consideriamo una richiesta astratta identificata come **R-42**.

Il controllo periodico osserva che **R-42** è pronta e che la versione corrente è **V7**.

Il sistema costruisce quindi un'identità logica equivalente a:

```
R-42:V7
```

Prima di creare una nuova consegna controlla se esista già un dispatch aperto con quella stessa identità.

Primo controllo

Non esiste nulla.

Il sistema crea il durable dispatch, lo assegna all'esecutore autorizzato e registra il claim.

Secondo controllo

La richiesta è ancora formalmente visibile, ma il dispatch **R-42:V7** esiste già ed è aperto.

Il risultato del controllo è un no-op: nessuna nuova attivazione cognitiva.

Completamento

L'esecutore conclude il lavoro. La source of truth del lavoro viene verificata secondo i criteri applicabili. Solo allora il dispatch può essere chiuso coerentemente.

L'esempio non introduce una nuova procedura WCM. Mostra soltanto, in forma semplificata, il significato degli invarianti già presenti nella baseline.

33.22 Relazioni con gli altri elementi WCM

PROT-004 non opera isolatamente.

La relazione più diretta è con **PROC-003 – Deterministic Discovery & Durable Dispatch**, che usa il protocollo nel passaggio tra discovery deterministica e attivazione dell'esecuzione.

```
PROC-003
scopre lavoro eleggibile
      ↓
PROT-004
rende la consegna durevole e idempotente
      ↓
EXECUTION
```

È inoltre collegato a **PROT-002 – Result Acceptance & Closure**, perché la terminalità del dispatch deve dipendere da un esito verificato e non dalla semplice fine di una run.

Più in generale, PROT-004 applica una distinzione ricorrente nel WCM:

```
STATO AUTOREVOLE DEL LAVORO
≠
MECCANISMO CHE LO CONSEGNA
```

Il protocollo protegge il confine tra questi due livelli.

33.23 Output del protocollo

L'output di PROT-004 non è soltanto “un esecutore è stato svegliato”.

Quando il dispatch è necessario, l'output corretto comprende una **consegna durevole e identificabile**, con:

- identità logica deduplicabile;
- riferimenti sufficienti al lavoro autorevole;
- destinazione coerente;
- claim persistente;
- stato del lifecycle ricostruibile.

Quando esiste già un dispatch aperto equivalente, l'output corretto può invece essere semplicemente:

```
NO-OP
```

Questa è una caratteristica importante del protocollo: **non fare nulla può essere il risultato corretto**, quando fare qualcosa produrrebbe una duplicazione.

33.24 Maturity e limiti

Il protocollo canonico è attualmente classificato **VALIDATED**.

La sua promozione deriva da evidenza operativa che ha mostrato sia failure mode del wake generico sia il comportamento corretto del dispatch durevole e deduplicato nel perimetro testato.

Questo non equivale a sostenere che ogni tecnologia, ogni runtime o ogni organizzazione implementino allo stesso modo il concetto di dispatch.

La baseline distingue il **principio generale WCM** dalla **specifica implementazione tecnica corrente**.

Il principio generale è:

- consegna persistente;
- contesto canonico;
- source of truth separata;
- deduplicazione persistente;
- claim;
- terminalità verificabile.

Dettagli come il meccanismo concreto di assignment, la forma della chiave atomica o l'oggetto tecnico usato come envelope dipendono dalle capacità del runtime validato.

PROT-004 non autorizza inoltre alcuna esecuzione che sia bloccata da governance, authority o gate applicabili.

33.25 La regola da ricordare

Se dovessimo conservare una sola idea di questo capitolo, sarebbe questa:

| Ripetere il controllo non deve significare ripetere la consegna.

Un lavoro pronto deve attraversare il confine verso l'esecuzione mediante un oggetto durevole che permetta di sapere:

- quale lavoro è;
- quale versione è stata osservata;
- dove deve essere risolto;
- chi lo ha preso in carico;
- se quella stessa consegna esiste già;
- quando può essere considerata realmente chiusa.

È così che il WCM trasforma un semplice “wake” in una consegna operativa verificabile.

Source Map

Fonti canoniche utilizzate per il Technical Truth Pass:

- `WCM_AGENT_START.md`;
- `wcm/documentation/process-memory-book/BOOK_INDEX.md`;
- `wcm/process-book/protocols/PROT-004_CANONICAL_DISPATCH_IDEMPOTENCY.md`;
- `wcm/process-book/processes/PROC-003_DETERMINISTIC_DISCOVERY_DURABLE_DISPATCH.md`;
- `wcm/process-book/protocols/PROT-002_RESULT_ACCEPTANCE_CLOSURE.md` per il solo confine acceptance/closure richiamato dal protocollo.

Maturity qualifier

`PROT-004` e `PROC-003` sono classificati **VALIDATED** nella baseline canonica corrente. Il capitolo usa tale stato nel perimetro documentato dalle fonti e non lo presenta come prova universale di efficacia, portabilità o completezza in ogni possibile ambiente operativo.